

instruction-design-pattern

Nazwa

Fasada (ang. Facade) - wzorzec strukturalny.

Problem

Podsystem składa się z dużej liczby klas. Korzystanie z takiego podsystemu może być trudne ze względu na konieczność znajomości szczegółów wielu klas.

Chcemy uprościć interfejs dla typowych przypadków użycia i zmniejszyć zależność między klientem a podsystemem.

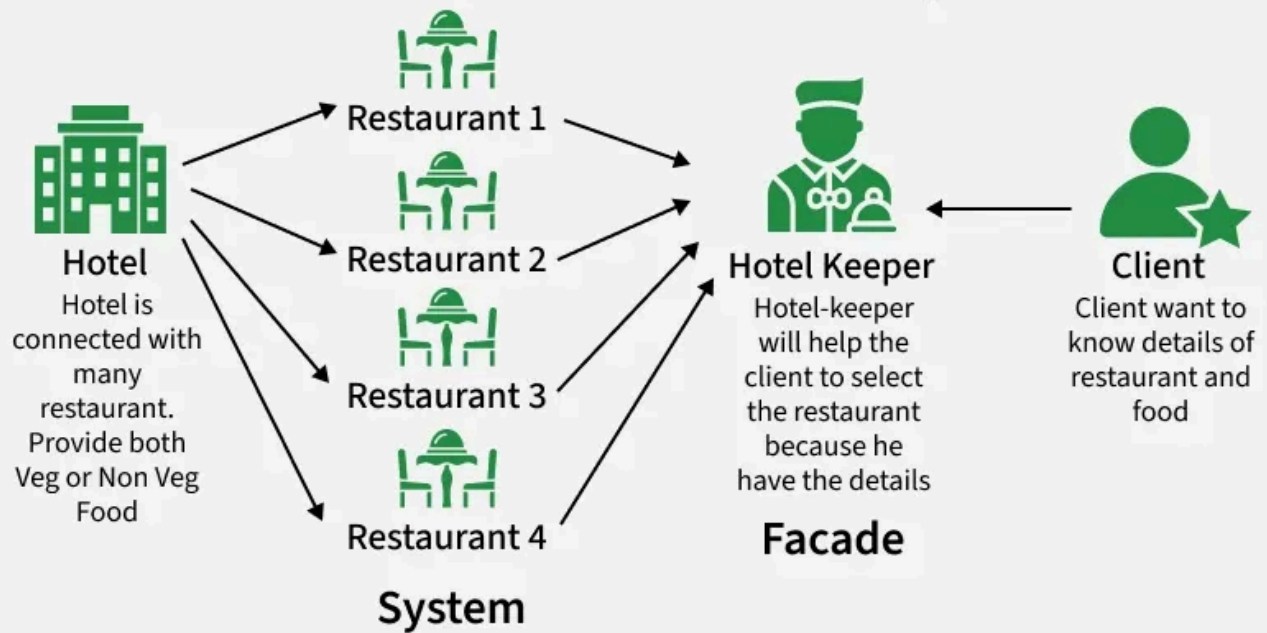
Rozwiązanie

Tworzymy Fasadę czyli dodatkową klasę, która zapewnia ujednolicony interfejs dla zestawu interfejsów w podsystemie.

- Fasada zna funkcje podsystemu i wie jak je użyć
- Klient nie musi wiedzieć jaka jest logika wewnątrz
- Fasada tylko zarządza wywołaniami, ale nie definiuje logiki
- Można mieć kilka fasad do tego samego podsystemu - np. jedną dla admina, drugą dla zwykłego użytkownika.

Przykładowo: hotel, w którym wiele restauracji serwuje różne rodzaje dań, takie jak dania wegetariańskie, mięsne i mieszane. Jako klient nie wchodzisz w bezpośrednią interakcję z każdą restauracją ani nie znasz ich menu. Zamiast tego komunikujesz się z właścicielem hotelu, który rozumie system i pobiera dla Ciebie potrzebne menu. Upraszcza to interakcję, zapewniając dostęp do wielu usług w jednym miejscu.

Problem Statement of Facade Method Design Pattern



Konsekwencje

Zalety:

- Uproszczony interfejs - klient nie musi znać szczegółów podsystemu.
- Loose coupling - klient jest niezależny od wewnętrznych klas.
- Zmiana implementacji podsystemu nie wpływa na kod klienta, jeśli Fasada zachowuje ten sam interfejs.
- Łatwiejsze testowanie i zrozumienie kodu.

Wady:

- Fasada może stać się *God Object* jeśli będziemy wrzucać do niej za dużo.
- Klient traci dostęp do zaawansowanych opcji podsystemu - fasada może nie udostępniać wszystkiego.
- Dodatkowa warstwa abstrakcji = więcej kodu do utrzymania.